

DICOM Overlays & Masking: Storage, Interpretation, Rendering

A reference module covering **burned-in annotations**, **retired overlays embedded in Pixel Data**, and **modern Overlay Plane groups (60xx,xxxx)**, including bit/byte layout, the rendering pipeline, toggling, and practical pitfalls. Companion file: [dicom_overlay_cornerstone_lab.html](#) (live Cornerstone demo with drawing).

1. What problem overlays solve

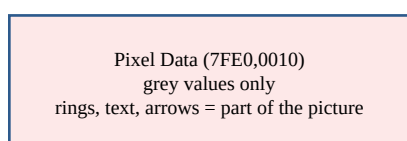
Radiology images are grey-level measurements (CT numbers, signal intensity). Often you also need **graphics on top**: a rectangle around a lesion, an arrow, a text label, a computer-aided-detection (CAD) mask, a region of interest, a laterality marker. If you write those marks into the grey values (pixels) you destroy diagnostic data and can never remove them. Overlays solve this by keeping **graphics separate from measurements**, so the viewer can show, hide, recolour or ignore them while the original pixels stay untouched.

Key point: An overlay is a second, 1-bit-per-pixel image ("is there a mark here?") stored with the image and drawn by the viewer *after* normal grey-scale processing. It has no grey value of its own; the viewer decides colour.

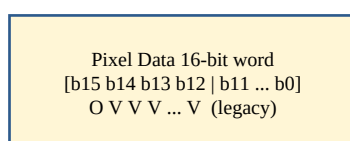
2. Three generations (and neighbours)

Method	Where the mark lives	Removable?	Status
A. Burned-in	Inside (7FE0,0010) Pixel Data: grey values are overwritten (text, arrows, markers, sometimes PHI).	No. Pixels are permanently altered.	Legal and still very common (ultrasound, secondary capture, screen grabs). Flag (0028,0301) Burned In Annotation = YES/NO.
B. Embedded overlay (legacy)	Spare high bits of each Pixel Data word, e.g. Bits Allocated 16, Bits Stored 12, overlay in bit 12. (60xx,0100)=16, (60xx,0102)=12.	Yes, by ignoring/masking the bit. But if you forget to mask it, the bit corrupts the grey value.	Retired in DICOM 2004. Still found in old CR/DX/MR archives. Must be read.
C. Overlay Plane (modern)	Own element (60xx,3000) Overlay Data, 1 bit/pixel, packed, group 6000 to 601E (even numbers).	Yes; fully independent of pixels.	Current. Plane Module PS3.3 C.9.2; Multi-frame C.9.3.
D. Neighbours	Shutters (0018,16xx); Graphic Annotation Seq. (0070,0001) in GSPS; Segmentation (SEG) objects; SR coordinates; Curves (50xx, retired).	Yes	Use for vector graphics, shared state, or labelled 3-D masks.

A. Burned-in



B. Embedded (retired)



C. Overlay Plane (modern)

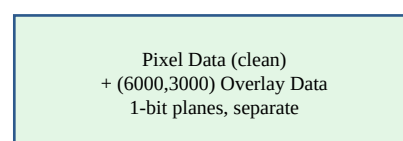


Figure 1. Where the mark is stored in each generation.

3. Burned-in annotations in depth

Here the mark is just pixels. If the original pixel was 1180 (soft tissue) and the text colour is 4095, the stored value 1180 is lost. Consequences: (1) window/level changes affect the text brightness like any other tissue; text may vanish at some windows; (2) the text rotates, zooms and flips with the image, so mirrored text appears after a flip; (3) it cannot be toggled; (4) **PHI risk**: patient names burned in ultrasound or secondary-capture images survive normal de-identification, so (0028,0301) must be checked and pixels cleaned (blackout rectangles) separately; (5) AI and measurement algorithms may read the text as anatomy.

Practical rule: if you can control the source, do **not** burn in. If you receive it, treat (0028,0301) = YES as a warning and apply pixel-level redaction for external sharing.

4. Modern Overlay Plane Module: the attributes

Each overlay uses an even group number 6000, 6002, ... 601E, so a single object holds at most **16 overlay planes**. Every attribute below exists in each active group; replace "60xx" by the group.

Tag	Name	VR	Meaning / typical value
(60xx,0010)	Overlay Rows	US	Height of the bitmap in pixels, e.g. 512
(60xx,0011)	Overlay Columns	US	Width of the bitmap, e.g. 512
(60xx,0015)	Number of Frames in Overlay	IS	Only multi-frame overlays; frames stacked in one bit stream
(60xx,0022)	Overlay Description	LO	Free text, e.g. "CAD detections"
(60xx,0040)	Overlay Type	CS	G = Graphics (annotation) or R = ROI (region measured/statistics)
(60xx,0045)	Overlay Subtype	LO	e.g. USER or AUTOMATED
(60xx,0050)	Overlay Origin	SS (2)	Row\Column of the overlay top-left relative to image top-left. 111 means aligned, 1-based. Can be 0 or negative for margins.
(60xx,0051)	Image Frame Origin	US	First image frame the overlay applies to (default 1)
(60xx,0100)	Overlay Bits Allocated	US	1 for modern planes; 16 for legacy embedded
(60xx,0102)	Overlay Bit Position	US	0 for modern planes; 12-15 for legacy embedded (which bit of the pixel word)
(60xx,1500)	Overlay Label	LO	Name shown in viewer lists
(60xx,1301-1303)	ROI Area/Mean/Std Dev	IS/DS	Statistics when type is R
(60xx,3000)	Overlay Data	OB/OW	The packed bitmap, see section 5

5. Bit-level and byte-level storage of Overlay Data

Rules (PS3.3 C.9.2, PS3.5): (1) one bit per overlay pixel, 1 = overlay on, 0 = off; (2) pixels are ordered **row by row, left to right, top to bottom** (row-major), starting with the top-left pixel; (3) rows are **not** padded to a byte boundary, the stream is continuous; (4) bits are filled into each byte starting at the **least significant bit** (bit 0) for little-endian transfer syntaxes; (5) total bits = Rows x Columns x Frames; (6) if the byte count is odd, one trailing zero byte makes the length even; (7) overlay data is **never compressed**, even when Pixel Data is JPEG or JPEG 2000.

```

pixel index  n    = row * Columns + col          (0-based)
byte offset  = n >> 3      (n div 8)
bit in byte  = n & 7      (n mod 8, 0 = LSB)
is_on        = (OverlayData[n >> 3] >> (n & 7)) & 1
bytes (no pad) = ceil(Rows * Columns * Frames / 8)
length (4 B VL) = bytes rounded up to even

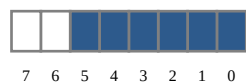
```

Worked example: Rows 4, Columns 5 (a 5x4 box outline)

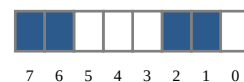
Overlay grid (number = bit index n)

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19

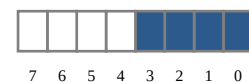
Byte 0 (bits 0..7)



Byte 1 (bits 8..15)



Byte 2 (bits 16..23)



= 0x3F (MSB..LSB shown left to right) = 0x0F (MSB..LSB shown left to right) = 0x0F (MSB..LSB shown left to right)

Byte 3 = 0x00 pad (stream 3 bytes, odd, so +1). Stored: 3F C6 0F 00

Element: (6000,3000) OB, VL = 4

Figure 2. Row-major, LSB-first packing. Note that the second row begins inside byte 0 (no row padding).

```

Raw element in Explicit VR Little Endian (hex):
00 60 00 30 4F 42 00 00 04 00 00 00 3F C6 0F 00
^group ^elem ^VR=OB ^rsvd ^VL=4 ^data (padded)

```

(tag stored as 16-bit LE group, then 16-bit LE element)

Key point: Because rows are not byte aligned, you cannot copy rows as bytes unless Columns is a multiple of 8. Always index by $n = r \times \text{Columns} + c$.

Legacy embedded planes (Retired)

Old CT/CR files store e.g. Bits Allocated 16, Bits Stored 12, High Bit 11. Bits 12-15 of every 16-bit pixel word are unused by the image, so vendors placed the overlay in bit 12. The overlay attributes say (60xx,0100)=16 and (60xx,0102)=12 and there is **no** (60xx,3000). The viewer must (a) take overlay = (word >> 12) & 1; (b) take grey = word & 0x0FFF (mask by Bits Stored). A viewer that forgets (b) shows values up to 65535 and a distorted window.

16-bit pixel word (BitsAllocated 16, BitsStored 12)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

b15..b12: unused / overlay bit (Overlay Bit Position)

b11..b0: grey value = word & 0x0FFF

Little-endian bytes: low byte = b7..b0, high byte = b15..b8. Overlay at b12 is bit 4 of the HIGH byte.

Figure 3. Legacy overlay hidden in the unused high bits of Pixel Data.

6. How a viewer interprets and renders overlays

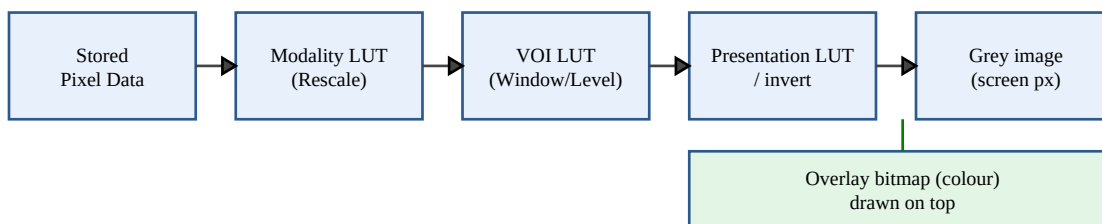


Figure 4. Overlays bypass the grey-value chain. They are composited after it.

Step by step. 1) Read (60xx,0010/0011) to get bitmap size. 2) Unpack the bits (section 5) into a 0/1 mask. 3) Place the mask: its top-left sits at image pixel (OriginRow-1, OriginCol-1), because origin is 1-based. 4) Apply the *same geometric transform as the image* (zoom, pan, rotation, flip), nearest-neighbour so mask pixels stay crisp. 5) Fill 1-bits with a viewer-chosen colour (white by default, or user colour, or per-plane colour; DICOM does not store colour in the plane; a GSPS can recommend a grey level). 6) Draw *after* VOI/Presentation LUT so window/level never alters it.

```
for r in 0..Rows-1:
    for c in 0..Cols-1:
        n = r*Cols + c
        if (data[n>>3] >> (n&7)) & 1:
            imgRow = r + (OriginRow-1); imgCol = c + (OriginCol-1)
            paint(screen(imgRow,imgCol), planeColour)
```

Multi-frame images

For multi-frame overlays (60xx,0015) = F, the bit stream is the concatenation of F frames. Image frame k (1-based) uses overlay frame k - ImageFrameOrigin + 1, and its first bit is $n0 = (\text{that frame index} - 1) \times \text{Rows} \times \text{Columns}$. A single-frame overlay on a multi-frame image applies to all frames (the common case for static marks).

Can overlays be toggled?

Yes, for B and C; no for A. The standard defines *storage*, not UI, but because planes are separate, every viewer can show/hide each of the 16 groups independently (and recolour). In a GSPS, the Overlay Activation Layer (60xx,1001) links an overlay to a graphic layer so its visibility follows the presentation state. Burned-in marks can only be hidden by loading a different object or cropping.

7. Practical usage

- **CAD/AI results:** detections as Overlay Type G, sub-type AUTOMATED. Better today: SEG, SR or GSPS if you need labels, per-object metadata or 3-D.
- **Legacy migration:** convert retired embedded overlays to 60xx planes: extract bit, mask pixels, set Bits Allocated 1/Bit Position 0, write (60xx,3000).
- **Archive/VNA ingest:** never recompress or reorder; validate (60xx,3000) length = even and $\geq \text{ceil}(R \cdot C \cdot F / 8)$.
- **De-identification:** inspect overlays for text (a text mask is a picture of letters!) and check (0028,0301). Group 60xx data is often missed by tag-only scrubbers.
- **Transfer syntax:** overlay data is always native; with Big Endian (retired) OW words are byte-swapped.

8. Common failure modes

Symptom	Likely cause
Overlay is a scrambled pattern	Assumed row padding to byte boundary, or MSB-first bit order
Overlay shifted by 1 px	Treated Origin 1\1 as 0\0
Image looks washed out, huge values	Legacy embedded bit not masked with Bits Stored
Overlay vanishes in one viewer	Viewer ignores odd groups or caps at 8 planes; or planes not enabled
Overlay wrong after rotate/flip	Drawn in screen space instead of image space
Overlay too blurry	Bilinear interpolation; use nearest neighbour
Text still visible after "removal"	It was burned in Pixel Data

9. Standards map

Topic	Reference
Overlay Plane Module	DICOM PS3.3 C.9.2
Multi-frame Overlay Module	PS3.3 C.9.3
Display shutter	PS3.3 C.7.6.11 (0018,1600 family)
Presentation state graphics	PS3.3 C.10.5 Graphic Annotation (0070,0001)
Burned In Annotation	(0028,0301) Burned In Annotation
Encoding of bit streams, OB/OW	PS3.5 section 8 and Annex
De-identification profiles	PS3.15 Annex E

Treat section numbers as pointers and confirm against the current edition at dicom.nema.org.